Edo Village in Unreal Engine 5.8: import and setup guide

How to get the kit, the plaza district, the town extension, the fields and the canal bank from Blender into an Unreal project, and how to set up lights, World Partition, instancing, collision, navigation and terrain blending.

> **Status.** This guide and `tools/kit/unreal_import.py` were written without access to Unreal Engine. Nothing in Unreal has been run yet. Expect to fix small things on the first run (names of import options and menu entries can differ slightly in 5.8). Section 5 lists the likely ones. Vegetation (trees, grass, bushes) is left to Unreal; the crops in the fields are kit meshes and optional.

---

1. What you get

Everything is in the repository branch `claude/unreal-mcp-server-access-xyax0u`, folder `kit/`.

Item	Where	Numbers
Module meshes with simple collision (`UCX_`)	`kit/export/modules/<collection>/SM_Edo_*.fbx`	153 files in 53 collections, Nanite-ready, metric scale, origin at the module pivot
Textures	`kit/textures/T_Edo_<Set>/{BaseColor, Normal_DX, ORM}.jpg`	17 sets, 173 MB of JPG. Normal maps are DirectX (Unreal's convention), ORM = R ambient occlusion, G roughness, B metallic
Ground meshes in world space	`kit/export/district/*.fbx`	plaza, district streets, town streets and trails, water, plazas, field soil (flat placeholder materials)
Heightmap for a Landscape	`kit/export/terrain/Edo_Heightmap_1009.png` (+ `.r16`, `heightmap_info.json`)	1009 x 1009, 1 m per quad, built from the surfaces your houses stand on
Village manifest	`kit/village/village_manifest.json`	237 houses, 42,285 module instances (world transforms), 108 lantern positions
Fields and canal bank	`kit/village/fields_manifest.json`	47,286 instances: crops, ridges, paddy bunds, curbs, cobbles, boulders
Clustered lights	`kit/village/lights_clustered.json`	30 lights merged from the 108 lanterns (radius 30 m)
Import script	`tools/kit/unreal_import.py`	imports textures, builds the master material, imports meshes, places everything
Planning data	`kit/village/town_layout.json`, `district_layout.json`, `town_context.json`	plots, streets, fields (useful for PCG and splines)

Triangle budget, all instanced and Nanite: the houses are about 4.2 M triangles in total; the fields are about 11 M (47k crop instances). The 101 house modules are shared by all 237 houses, so the memory cost is the 153 meshes, not the instance count.

2. Before you start

1. **Engine and plugins.** Unreal Engine 5.8. Enable *Python Editor Script Plugin* and *Editor Scripting Utilities* (Edit > Plugins), restart.
2. **Project settings.** Nanite on (default). Lumen for global illumination and reflections, Virtual Shadow Maps (defaults of a new UE5 project). *Support Hardware Ray Tracing* is optional.
3. **Get the files.** Pull the branch (GitHub Desktop or `git pull`). Keep the `kit/` folder together. The repository is large (about 950 MB); the `kit/export` and `kit/textures` folders are what Unreal needs.
4. **Open a new level** (or your existing village level). Do the terrain first (section 4), then the import.

3. Coordinates (read this once)

Blender is Z up, right handed, metres. Unreal is Z up, left handed, centimetres.

```
Unreal X = Blender X * 100
Unreal Y = -Blender Y * 100
Unreal Z = Blender Z * 100
Unreal yaw = -(Blender rotation about Z, degrees)
```

North (+Y in Blender) is -Y in Unreal. The manifests and the heightmap are already converted this way by the scripts. **Check once:** the plaza centre is Blender (-48, -122) = Unreal (-4800, +12200). Spawn a cube there after the import; it should sit in the middle of the plaza.

4. Step 1: terrain (Landscape)

Your Blender terrain is a mesh (`EDO_Terrain`) with flat patches (`EDO_Ground_Town`, `EDO_Ground_Village`) that the houses stand on. The heightmap takes its height from those patches first, then from the terrain, so the houses meet the ground exactly.

1. Landscape mode > *New* > *Import from File*. Heightmap file: `kit/export/terrain/Edo_Heightmap_1009.png` (or the `.r16`, little endian).
2. Overall resolution **1009 x 1009**. Section size **63 x 63 quads**, **1 section per component**, **16 x 16 components**.
3. Location **X = -40000, Y = -14000, Z = 0**. Scale **X = 100, Y = 100, Z = 173**. (These come from `heightmap_info.json`: the region is Blender x -400..608, y 140..-868; Z scale 173 because the mountains reach 440 m.)
4. Create the Landscape. Check the plaza and the canal against the manifest (section 3).

If you already have a Landscape: keep it, make sure it uses the same world origin, and check that the houses sit on it. Where the two differ, use the Landscape *Flatten* tool or a Landscape patch under each house plot (section 12).

To regenerate the heightmap after changing the terrain in Blender: `python -I tools/kit/export_heightmap.py <village.blend> kit/export/terrain` (about 3 minutes).

5. Step 2: run the import script, and fix what it finds

1. Open `tools/kit/unreal_import.py` in a text editor. Set `KIT_DIR` to the full path of the `kit` folder.
2. **First run, no placement:** leave `PLACE_VILLAGE = False`, `PLACE_FIELDS = False`. This imports textures and meshes, builds the master material and the material instances, and imports the ground meshes. In Unreal: *Tools* > *Execute Python Script*, pick the file.
3. Watch the **Output Log** (filter by `LogPython`) and read every warning. The script logs `EdoKit: N meshes, N material instances` at the end of the first part; expect 153 meshes and 17 material instances.

Settings at the top of the script:

Setting	Default	Meaning
`DEST`	`/Game/EdoKit`	where assets go (`Textures`, `Materials`, `Meshes/<collection>`, `Meshes/District`)
`IMPORT_TEXTURES`, `IMPORT_MESHES`, `IMPORT_DISTRICT_GROUND`	True	switch a part off when it is already imported
`PLACE_GROUND`	True	put the ground meshes at the world origin (they are exported in world space)
`PLACE_VILLAGE`	False	place houses from the manifest
`HOUSES`	`['DISTRICT_*', 'TOWN_*', 'BRIDGE_*', 'PROP_*']`	which manifest entries to place (`fnmatch`, `None` = all, e.g. `['TWN_1*']`)
`PLACE_AS`	`'instances'`	hierarchical instanced meshes per module and cell (about 1,200 actors for all houses) or `'actors'` (42,285 actors, slow)
`CELL_M`	128	cell size of the instanced actors, the default World Partition cell
`LANTERN_LIGHTS`	`'clustered'`	30 real lights, `'all'` = 108, `False` = none
`PLACE_FIELDS`	False	crops and canal bank from `fields_manifest.json` (always instanced)

Likely first-run problems

Message or symptom	Cause and fix
`import option xyz: ...` warnings	an FBX import option has another name in 5.8. Open one mesh import dialog by hand, find the option (Build Nanite, Combine Meshes, Auto Generate Collision off, Generate Lightmap UVs off) and rename it in the script's `for k, v in (...)` list
`moss layer skipped: ...`	a material expression class or property name differs. The material still works without moss. Add the moss blend by hand: lerp BaseColor with `MossColor` by (up-facing normal Z, noise) x `MossAmount`
`texture import failed`	wrong path in `KIT_DIR`, or a missing set. Check `kit/textures/T_Edo_*/`
meshes import but the material slots are grey	the instance name must equal the slot name (`M_Edo_Timber` slot = `MI_Edo_Timber` instance): the script maps by the slot name. If a name differs, assign by hand or fix the mapping in the `for i, slot in enumerate(...)` loop
`no mesh for module X`	module FBX not imported (check the log above) or the module is new: re-export with `export_fbx.py`
collisions missing	the FBX must contain `UCX_<mesh>_NN` objects. If Unreal ignored them, re-import with *Auto Generate Collision* off and check *Static Mesh* > *Collision* > *Complexity*
`instanced placement failed`	`add_component_by_class` is not exposed this way in 5.8. The script falls back to one actor per module; for the fields, use the manifest with PCG or a small Blueprint (section 9)
the editor freezes during placement	normal: it creates thousands of objects. Place in batches (`HOUSES = ['DISTRICT_*']`, then `['TOWN_*']`, ...), save between batches
everything is mirrored	the Y flip: check `to_ue()` in the script and the Landscape location (section 3, 4)

Check list after the first run

- Content Browser: `/Game/EdoKit/Meshes` has 53 sub folders; open `SM_Edo_Wall_A_Plaster` or a roof module: Nanite enabled, one `UCX_` hull in *Collision*.
- Open `MI_Edo_Timber`: the three textures are assigned; `MI_Edo_Lantern`: `EmissiveStrength` is 6.
- Place the test set: `PLACE_VILLAGE = True`, `HOUSES = ['DISTRICT_HALL']`, `IMPORT_*` False. The hall (13 x 8 bays, two storeys) should look like

`kit/previews/district_hall_front.jpg` and `district_plaza_hall.jpg`. Compare materials with the Blender previews.

- Then place everything: `HOUSES` as in the table above. Compare the top view with `kit/previews/village_topdown_wide.jpg`.

6. Materials and textures

- **Master material** `M_EdoKit_Master`, one instance per texture set (`MI_Edo_Timber`, `Planks`, `Plaster`, `Kawara`, `Thatch`, `Stone`, `StoneWall`, `Paper`,

`Ceramic`, `Cloth`, `ClothRed`, `Lantern`, `Iron`, `Leaf`, `Grain`, `Soil`, `Moss`). Parameters: `BaseColor`, `Normal`, `ORM`, `MossColor`, `MossAmount` (0 to 0.5, per set), `EmissiveStrength` (the lantern paper uses 6).

- **Texture settings** (the script sets them): BaseColor sRGB, Normal_DX as *Normal map* (no green flip), ORM as *Masks* with sRGB off.
- **Two sided:** the leaf, grain and red cloth meshes (crops, stall awnings, parasol) are modelled double sided, so the one-sided master works. If you see

holes from behind, open the instance, *Base Property Overrides* > *Two Sided* on.

- **Texture memory:** 17 sets x 3 maps, most at 4K. Plan for 0.5 to 1 GB of streamed texture memory. Set `r.Streaming.PoolSize` (3000 or more on a PC) or turn

on *Virtual Texture Streaming* for the 4K sets (Timber, Planks, Plaster, Kawara, Thatch, StoneWall).

- **Moss and weathering** are material effects: the moss blend is world-noise masked and only on up-facing surfaces; raise `MossAmount` per instance for

older houses. For variation between houses use a per-instance custom data float (the manifest gives you the house id) or vertex colour.

7. Lights: replace the ~100 point lights with emissive + a few real lights

The village has 108 lantern positions (door lanterns of shops, restaurants, inns, tea houses, the hall and the hospital). 108 point lights are expensive with Virtual Shadow Maps and add little.

1. **The glow is the material.** `MI_Edo_Lantern` is emissive (paper). With Lumen the emissive paper lights the wall around it. Keep it.
2. **A few real lights for warmth and long reach.** `python tools/kit/cluster_lights.py 30` merges the lanterns within 30 m into 30 lights (`kit/village/lights_clustered.json`). `python tools/kit/cluster_lights.py 45` gives 19. The import script (`LANTERN_LIGHTS = 'clustered'`) places them with intensity 6 x sqrt(count) and a reach of 4 m + 1.2 x the cluster radius, shadows off.
3. **Hero lights by hand:** the plaza (a warm point light above the well), the gates, the hospital court, the shrine later. Only these should cast shadows.
4. **Night look:** exponential height fog with a little inscattering, a low sky light, a dim warm directional moonlight. Lantern emissive 6 to 12.
5. **Check:** `stat gpu`, then *Show* > *Visualize* > *Light Complexity*; keep the number of lights per pixel low in the streets.

8. World Partition

1. Create the level as *Open World*, or convert it (*File* > *Convert Level to World Partition*). Keep *One File Per Actor* on.
2. **Grid:** the default runtime grid cell is 12,800 cm (128 m). The import script groups its instanced actors per module in 128 m cells (`CELL_M`), so each cell streams as a unit. For this village (about 570 x 470 m plus the hills) a single grid with a loading range of 250 to 400 m is enough.
3. **Actor counts:** all houses = about 1,183 instanced actors (module x cell); the fields and canal bank add about 50 more. Fine for World Partition.
4. **Data layers** (create in the Data Layers outliner, assign by folder): `Houses` (`EdoVillage/Houses`), `Fields` (`EdoVillage/Fields`), `Lights` (`EdoVillage/Lights`), `Ground` (`EdoVillage/DistrictGround`). Useful to switch fields and lights off while you work.
5. **HLOD:** add an HLOD layer for the houses (instancing or merged mesh) so distant houses cost little; the roofs and walls are already Nanite, so HLOD matters most for the lights and the crop instances.
6. **Always loaded:** put the Landscape, the sky, the post process volume and the NavMeshBounds in the persistent (non-spatially-loaded) layer.

9. Instanced meshes for the crops (and everything else)

- The houses and the fields are placed as **hierarchical instanced static meshes**: one component per module and cell, thousands of instances each. This is

what `PLACE_AS = 'instances'` and `PLACE_FIELDS = True` do.

- **Budget:** the fields are 11 M triangles (47,286 instances). Nanite handles this, but set a **cull distance** on the crop components (for example 150 to

250 m, *Cull Distance* / *Min/Max Draw Distance*) and consider fewer crop meshes in the far fields. The ridges, bunds and canal bank are cheap.

- **Alternatives:** (a) a PCG graph that scatters `SM_Edo_CropCabbage_*`, `CropBarley_*`, `CropRice_*` inside the field patches (`town_layout.json` has

centre, size and crop for each of the 13 fields), (b) the Foliage tool, (c) your own vegetation system. Crops are not vegetation in the sense of your trees and grass; drop them if your system does it better.

- **If the script cannot create instanced components in 5.8:** read `fields_manifest.json` (rows `[x, y, z, yaw_deg, sx, sy, sz]` in Blender metres per module)

in a small editor utility Blueprint and call *Add Instances* on a `HierarchicalInstancedStaticMeshComponent`.

10. Collision

- **What exists:** every module FBX carries one convex hull (`UCX_`, at most 120 vertices) except thin or hanging things (noren, lanterns, balcony brackets, shop signs,

laundry poles, drying racks, herb racks, sword and axe racks, crops, ridges, paddy bunds, cobble tiles). The gate and the two timber bridges use boxes; the stone arch bridge uses 14 convex columns so the arch stays open for the water and the deck is walkable.

- **Check in the viewport:** *Show* > *Collision* (or *Player Collision*) and walk the streets with a test character. Things to check: the walls of the houses

(no walking through), the stilts and footing stones (blocking, but the space between is open), the engawa steps (14 risers of 19 cm: the default step height of 45 cm is fine), the gates, the bridges, the canal curb (blocks) and cobbles (walkable, no collision of their own: the Landscape is under them).

- **Complexity:** set *Collision Complexity* to *Use Simple And Complex* only where you need trace precision; simple hulls are cheaper for character movement.
- **Known limits:** doors and windows are closed (no interiors yet), so houses are solid boxes; roofs of the big buildings use one hull per module.

11. Navigation mesh

1. Add a **NavMeshBoundsVolume** covering the village, the fields and the streets towards the trail (about 600 x 500 m); with World Partition keep it in the persistent layer and set *Runtime Generation* to *Dynamic* in Project Settings > Navigation Mesh (or use the World Partition navmesh options of 5.8).
2. Rebuild navigation (Build > Build Paths), then press **P** in the viewport. Check: streets, plaza, bridges, the gate openings and the engawa steps are walkable; the canal and the river are not; the inside of the houses is not.
3. **Areas:** add a *NavModifierVolume* with a *Null* area over the canal and river, and a cheaper custom area (or *NavArea_Default* with lower cost) over the streets so AI prefers them.
4. **Agent size:** radius about 34 cm and height 176 cm for a human; the narrowest opening is the roofed gate (1.7 m between the piers).
5. If the nav mesh has holes under the stilted houses, that is expected (the floor is 1.3 m above the ground). If you want people to walk under the engawa, it is open there.

12. Terrain blending (houses, roads, plaza, canal bank, fields)

The kit meshes stand on the terrain; blending is a Landscape material and splines job.

1. **Landscape material** with layers: grass, soil/mud, gravel, cobble, mossy rock, field soil (use `T_Edo_Soil` and `T_Edo_Stone` as a start).
2. **Streets and plaza:** the ground meshes (`SM_Edo_District_Roads`, `SM_Edo_Town_Roads`, `SM_Edo_Town_Plazas`) are flat ribbons 3 to 6 cm above the terrain, so they z-fight or float on slopes. Best: paint the street layer on the Landscape and flatten with **Landscape splines** (or a *Landscape Patch*) along the street curves (`district_layout.json` and `town_layout.json` list them with widths), then drop the ribbon meshes. Second best: keep the meshes and give their material **Virtual Texture Output** so they blend into the Landscape through a *Runtime Virtual Texture*.
3. **Houses:** footing stones and stilts are the transition. Add a Landscape layer *dirt* under each plot (a spline loop or a decal at the plot corners in `town_layout.json`) and a **Runtime Virtual Texture** blend on the footing stones (`SM_Edo_Footing_*`, material `MI_Edo_Stone`) so the bottom edge fades into the ground.
4. **Canal bank:** the curb and cobble tiles sit on the terrain by raycast. Give `MI_Edo_Stone` / `MI_Edo_StoneWall` RVT output on the cobble tile only (not the walls) so the paving melts into the soil; keep a soil layer under the cobbles.

5. **Fields:** paint a field-soil layer under each patch (`SM_Edo_Town_FieldSoil` shows where); the crop ridges (`SM_Edo_CropRidge`) sit on top. Paddy water: replace `SM_Edo_Water` (flat placeholder) with the **Water plugin** or a water material; the paddy bunds hold it.
6. **Where the flat town ground meets the Landscape hills** (east of x = 62) the heightmap may show a seam: smooth it with the Landscape *Smooth* tool.

13. Performance and quality checklist

- `stat unit`, `stat gpu`, `stat nanite`; *Show* > *Visualize* > *Nanite Visualization* > *Overview*. Expect heavy Nanite use in the fields.
- Lumen: *Scene Detail* high, *Final Gather Quality* 1 first; Virtual Shadow Maps on; shadow cache on. The village is mostly static.
- Many small lights are the main risk: keep them clustered (section 7).
- Streaming: check *Texture Streaming Pool over budget* (`stat streaming`). Raise the pool or use Virtual Textures.
- Keep the crop cull distance short (section 9). Hide `Fields` while you work.

14. Updating after changes in Blender

Run in your village `.blend` (headless commands shown; in Blender use the Text Editor):

```
python -I tools/kit/build_kit.py kit/Edo_Kit.blend                    # rebuild the kit, node groups and layo
python -I tools/kit/install_kit.py <village.blend> kit/Edo_Kit.blend --save <village_kit.blend>
python -I tools/kit/village_build.py <village_kit.blend> kit/Edo_Kit.blend kit/village      # manifests
python -I tools/kit/export_district_ground.py <village_kit.blend> kit/export/district
python -I tools/kit/export_fbx.py kit/Edo_Kit.blend kit/export                              # only when a modu
python -I tools/kit/export_heightmap.py <village.blend> kit/export/terrain                  # only when the te
python  tools/kit/cluster_lights.py 30
```

Then set the `IMPORT_*` and `PLACE_*` flags in `unreal_import.py` for what changed (re-importing replaces existing assets) and delete the old placed actors (select the folder `EdoVillage/...` in the World Outliner) before placing again.

15. Troubleshooting

Symptom	Likely cause	Fix
Houses float or sink	Landscape differs from the Blender ground	regenerate the heightmap, or flatten under the plots; check the Z scale 173
Whole village mirrored or rotated	Y flip or yaw sign	`to_ue()` in the script uses (x, -y, z) and yaw = -rz
Roofs look flat / black	normal map sRGB on, or green flipped	Normal_DX must be *Normal map*, sRGB off, no flip
Everything too shiny or too dull	ORM channels wrong	R = AO, G = roughness, B = metallic (only `T_Edo_Iron` is metallic)
Crops look dark	Leaf set too dark in the sun	raise the BaseColor multiplier on `MI_Edo_Leaf` / `MI_Edo_Grain`
Moss too strong or missing	`MossAmount` per instance	0 = off; Kawara 0.5, Stone 0.38, Timber 0.1 are the Blender values
Light leaks inside houses	closed shells are thin	add a ceiling plane or enable *Lumen Surface Cache* tweaks; interiors are not modelled yet
Editor is slow with all actors	too many actors	`PLACE_AS = 'instances'`, data layers, cull distances

16. What comes next

The next additions (shrine, temple, watchtower, gates, samurai quarter, watermill, terraces and more) are laid out on the planning map `kit/previews/village_additions_plan.jpg` (data in `tools/kit/additions.py`). Nothing on it is built yet. Do the Unreal import and the checks above first, so problems in the pipeline show up before more content

is added.